

HDI Predictor Using Machine Learning

Project Description:

HDI Predictor Using Machine Learning is a data-driven web application that estimates a country's Human Development Index (HDI) from key socio-economic indicators. The platform includes a Prediction module, which takes user-provided values — life expectancy, expected years of schooling, mean years of schooling, and GNI per capita — and returns a predicted HDI score, and a Categorization module, which classifies the predicted score into Low, Medium, High, or Very High development bands for easy interpretation.

Built with Flask and trained on real UN Human Development Report (HDR) 2025 data using Linear Regression, the application processes user inputs through a preprocessing pipeline before generating fast, reliable predictions. The trained model is serialized with joblib as HDI.pkl and loaded once at application startup for efficient inference. With a clean, responsive interface built in HTML and CSS, HDI Predictor helps students, researchers, and policy enthusiasts explore development indicators with confidence.

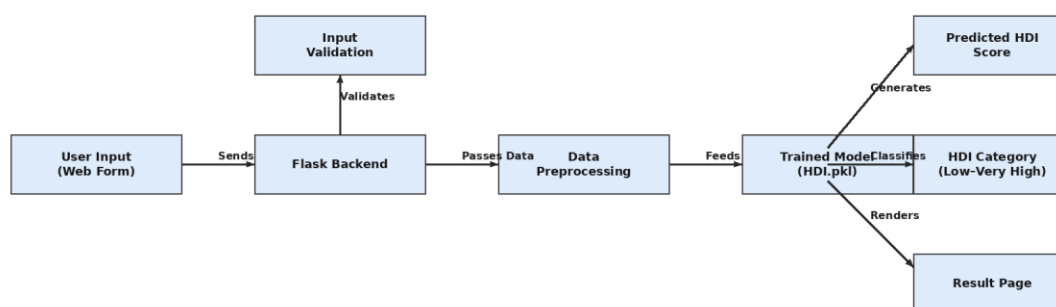
Scenarios

Scenario 1: A user enters life expectancy, education, and income indicator values for a country profile. The system validates and preprocesses the inputs, feeds them into the trained Linear Regression model, and returns a predicted HDI score along with its development category in under a second.

Scenario 2: A student researching development economics enters real indicator values for a lower-income country to see how the model classifies it. The application displays the predicted HDI score and labels it as “Low” or “Medium” development on a dedicated results page.

Scenario 3: A user submits an invalid indicator value (for example, a negative life expectancy). The Flask backend's input validation detects the invalid entry and returns a clear error message asking the user to correct it before a prediction is generated.

Technical Architecture:



Description: HDI Predictor utilizes a straightforward two-tier architecture to deliver fast, reliable predictions. The frontend provides a simple, responsive interface built with HTML and CSS for entering indicator values. The Flask-based backend validates and preprocesses these inputs before passing them to the trained Linear Regression model. The model, serialized with joblib and loaded once at startup, returns a predicted HDI score, which the backend then classifies into a development category before rendering the result page.

(Note: This project does not use a database, so no ER diagram is required.)

Pre-requisites:

Flask Framework Knowledge: [Flask Documentation](#)

Scikit-learn Familiarity: [Scikit-learn Documentation](#)

HTML & CSS Skills: [W3Schools HTML/CSS Tutorials](#)

Python Programming Proficiency: [Python Documentation](#)

Pandas & NumPy: [Pandas Documentation](#)

Version Control with Git: [Git Documentation](#)

Development Environment Setup: [Flask Installation Guide](#)

Project Workflow:

Milestone 1: Data Collection and Model Selection

- Activity 1.1: Collect and review the real UN HDR 2025 dataset (HDI.csv).
- Activity 1.2: Explore and visualize the dataset to understand feature distributions and their relationship with HDI.
- Activity 1.3: Preprocess the data — handling missing values and splitting into feature (X) and target (y) sets.
- Activity 1.4: Select Linear Regression based on the dataset's largely linear relationships and the need for interpretability.

Milestone 2: Model Training and Evaluation

- Activity 2.1: Split the dataset into training and testing sets.
- Activity 2.2: Train the Linear Regression model on the training data.
- Activity 2.3: Evaluate performance using the R^2 score, achieving ≈ 0.96 on the test set.
- Activity 2.4: Serialize the trained model using joblib as HDI.pkl for use in the Flask application.

Milestone 3: app.py Development

- Activity 3.1: Write the main application logic in app.py, establishing routes for the home page and the prediction endpoint, and loading the trained model at startup.

Milestone 4: Frontend Development

- Activity 4.1: Design the user interface using HTML and CSS, including an input form (index.html) and a results page (result.html).
- Activity 4.2: Implement server-side rendering with Flask's render_template to display the predicted score and category.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by installing dependencies from requirements.txt.
- Activity 5.2: Run the Flask development server locally and verify predictions across a range of indicator values.

Milestone 6: Conclusion

Milestone 1: Data Collection and Model Selection

In this milestone, the focus is on collecting and understanding the dataset used to train the HDI prediction model. This involves sourcing real Human Development Report (HDR) 2025 data, exploring feature relationships through visualization, and preparing the data for training.

Activity 1.1: Collect the Dataset

The dataset (HDI.csv) was sourced from the UN Human Development Report 2025 and contains indicators such as life expectancy, expected years of schooling, mean years of schooling, and GNI per capita for a range of countries.

Activity 1.2: Explore and Visualize the Data

Using Pandas and Matplotlib, the dataset was explored to understand distributions, correlations, and outliers, including scatter plots of each indicator against HDI to assess linear relationships.

```
import pandas as pd

df = pd.read_csv("HDI.csv")
df.describe()
df.corr()
```

Activity 1.3: Preprocess the Data

Missing values were handled, and the dataset was split into feature (X) and target (y) variables in preparation for model training.

```
X = df[["Life_Expectancy", "Expected_Years_Schooling",
        "Mean_Years_Schooling", "GNI_Per_Capita"]]
y = df["HDI"]
```

Activity 1.4: Select the Modeling Approach

Linear Regression was selected for its interpretability and strong fit with the dataset's largely linear relationships between the indicators and HDI.

Milestone 2: Model Training and Evaluation

Milestone 2 covers training the Linear Regression model on the prepared dataset and evaluating how well it predicts HDI on unseen data.

Activity 2.1 & 2.2: Split the Data and Train the Model

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
```

Activity 2.3: Evaluate Model Performance

The trained model was evaluated on the held-out test set using the R^2 score, a standard metric for regression accuracy.

```
from sklearn.metrics import r2_score

predictions = model.predict(X_test)
print("R2 Score:", r2_score(y_test, predictions))
# R2 Score: 0.96
```

An R^2 score of approximately 0.96 indicates that the model explains about 96% of the variance in HDI based on the four input indicators — a strong fit for this dataset.

Activity 2.4: Serialize the Trained Model

The trained model was serialized using joblib so it can be loaded directly by the Flask application without retraining.

```
import joblib

joblib.dump(model, "HDI.pkl")
```

Milestone 3: app.py Development

Milestone 3 focuses on creating the core logic of the app.py file, which serves as the backbone of the HDI Predictor application. This includes loading the trained model at startup, defining the Flask routes, and returning predictions to the frontend.

Activity 3.1: Writing the Main Application Logic

Load the trained model once when the application starts:

```
import joblib

model = joblib.load("HDI.pkl")
```

Define the home page route, which renders the input form:

```
@app.route("/")
def index():
    return render_template("index.html")
```

Define the prediction route, which processes form input and returns the result:

```
@app.route("/predict", methods=["POST"])
def predict():
    life_exp = float(request.form["life_expectancy"])
    edu_exp = float(request.form["expected_schooling"])
    edu_mean = float(request.form["mean_schooling"])
    gni = float(request.form["gni_per_capita"])

    features = [[life_exp, edu_exp, edu_mean, gni]]
    prediction = model.predict(features)[0]
    category = classify_hdi(prediction)

    return render_template("result.html",
        hdi_score=round(prediction, 3), category=category)
```

A helper function classifies the predicted score into a development band:

```
def classify_hdi(score):
    if score < 0.550:
        return "Low"
    elif score < 0.700:
        return "Medium"
    elif score < 0.800:
        return "High"
    else:
        return "Very High"
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, user-friendly interface for HDI Predictor. This involves designing a responsive input form and a clear results page using HTML and CSS.

Activity 4.1: Designing and Developing the User Interface

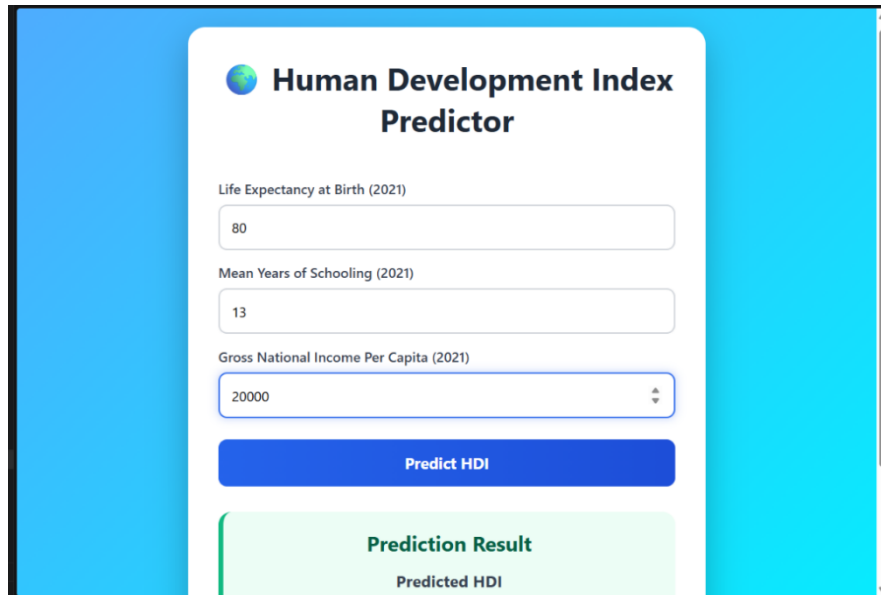
- Develop the main input page (index.html) with a form for life expectancy, expected years of schooling, mean years of schooling, and GNI per capita.
- Use semantic HTML elements to keep the structure organized and accessible.
- Style the form with a dedicated stylesheet (static/css/style.css) for a clean, responsive layout across desktop and mobile.

Activity 4.2: Result Rendering

- Create result.html to display the predicted HDI score and its category (Low, Medium, High, or Very High).
- Use Flask's `render_template` to pass `hdi_score` and `category` from the backend into the results page.
- Show the classification bands alongside the result so users can see how the score was categorized.

Exploring the Website's Web Pages:

Home / Input Page:

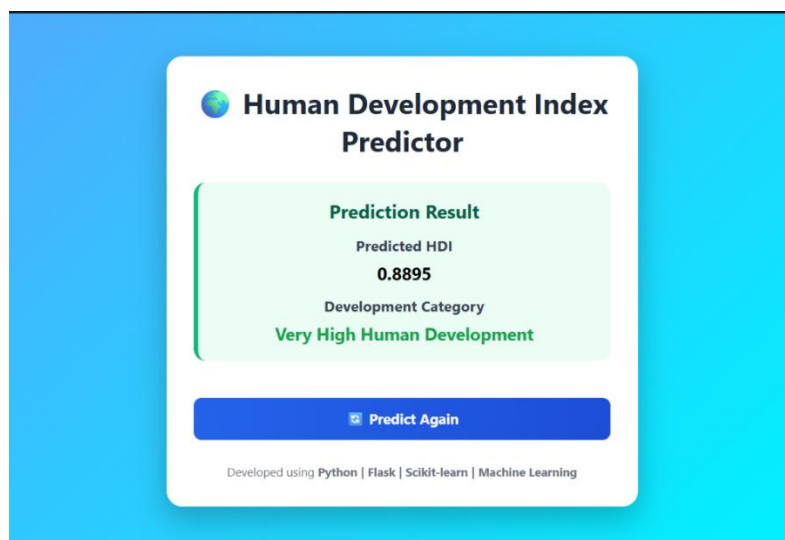


The mockup shows a web page with a blue background. In the center is a white card titled "Human Development Index Predictor" with a globe icon. Below the title are three input fields: "Life Expectancy at Birth (2021)" with the value 80, "Mean Years of Schooling (2021)" with the value 13, and "Gross National Income Per Capita (2021)" with the value 20000. Below these fields is a blue button labeled "Predict HDI". At the bottom of the card is a green box labeled "Prediction Result" with the text "Predicted HDI" below it.

Illustrative mockup — actual styling may vary

Description: The home page presents a simple input form where users enter the four HDI indicators — life expectancy, expected years of schooling, mean years of schooling, and GNI per capita. A single “Predict HDI” button submits the form to the /predict route.

Result Page:



The mockup shows a web page with a blue background. In the center is a white card titled "Human Development Index Predictor" with a globe icon. Below the title is a green box labeled "Prediction Result". Inside this box, it says "Predicted HDI" followed by the value "0.8895", and "Development Category" followed by the text "Very High Human Development". Below the green box is a blue button labeled "Predict Again" with a refresh icon. At the bottom of the card, it says "Developed using Python | Flask | Scikit-learn | Machine Learning".

Illustrative mockup — actual styling may vary

Description: The result page displays the predicted HDI score alongside its development category (Low, Medium, High, or Very High), along with the classification bands used, so the user can see exactly how the score was interpreted.

Milestone 5: Deployment

Milestone 5 focuses on deploying HDI Predictor locally to verify correct operation. As the project runs entirely on a local Flask server, no external hosting or tunneling service is required.

Activity 5.1: Preparing the Application for Local Deployment

Create and activate a virtual environment, then install all required dependencies from requirements.txt:

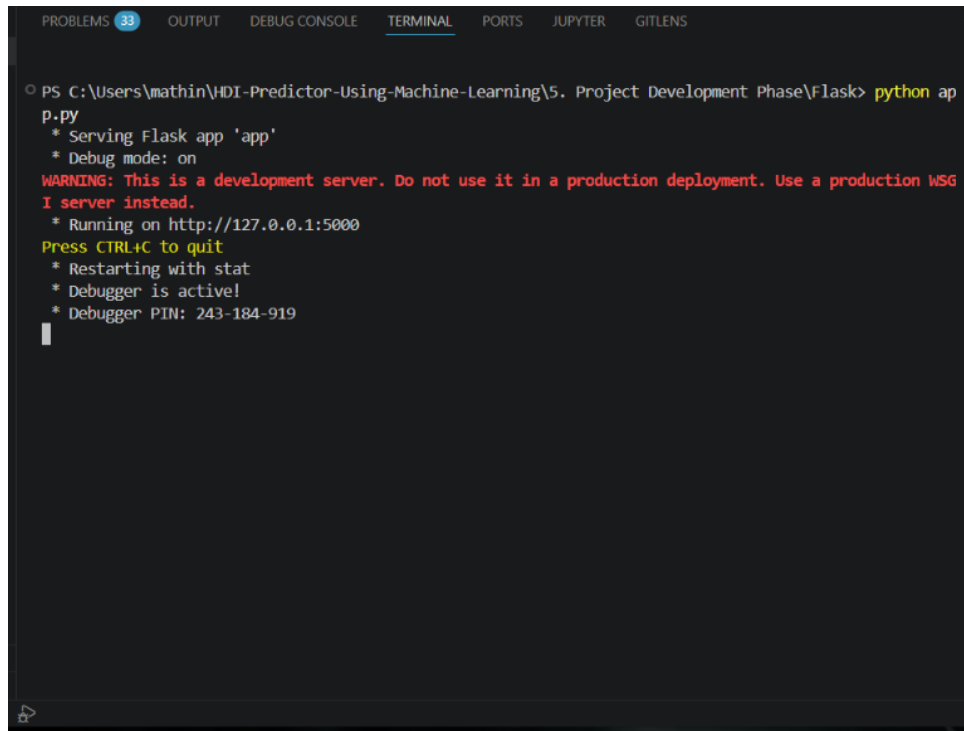
```
D:\HDI-Predictor>python -m venv myenv
D:\HDI-Predictor>"myenv\Scripts\activate"
(myenv) D:\HDI-Predictor>pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

Run the application locally using:

```
(myenv) D:\HDI-Predictor>python app.py
```

Once running, open a browser and navigate to <http://127.0.0.1:5000> to access the app:



```
PROBLEMS 33 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS
PS C:\Users\mathin\HDI-Predictor-Using-Machine-Learning\5. Project Development Phase\Flask> python ap
p.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
I server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 243-184-919
```

Illustrative terminal output — actual output may vary

Submit a range of indicator values through the input form to verify that predictions are generated correctly and classified into the right development category.

Conclusion:

HDI Predictor Using Machine Learning is a Flask-based web application that estimates a country's Human Development Index from four key socio-economic indicators. It uses a Linear Regression model trained on real UN HDR 2025 data, achieving an R^2 score of approximately 0.96, to generate fast and reliable predictions. The project, which included data exploration, model training and serialization, and a straightforward Flask frontend, demonstrates how a focused, interpretable machine learning model can turn raw development indicators into an accessible prediction tool. Looking ahead, HDI Predictor offers opportunities for expansion, such as deploying the application to a cloud platform, comparing additional regression models, and extending the dataset to cover more recent or more granular indicators. By continuing to refine the model and interface, the platform is well-positioned to grow from a course project into a broader educational tool for exploring human development data.